UNIVERSITY PROJECT

Chapter 2: Process Models

Based on Software Engineering: A Practitioner's Approach (9th Edition)

Authors: Roger S. Pressman and Bruce R. Maxim

Student Name: Asal Ghafouri

Instructor: Engineer Rasoulzadeh

Course: Software Engineering

1. Introduction

A software process is a structured set of activities, actions, and tasks used to develop and maintain a software product. A process model provides a simplified representation of that process. It helps a development team understand what work must be done, when it should be done, and how the activities relate to one another.

There is no single process model that is suitable for every project. The best choice depends on factors such as project size, complexity, risk, customer involvement, schedule, and how clearly the requirements are understood.

2. The Software Process

A software process describes the work required to build, deliver, and support software. It is not simply writing code. It also includes understanding the problem, planning the work, designing a solution, checking quality, delivering the product, and responding to changes.

A good process gives a team a common way to organize work while allowing the team to adapt when project conditions change.

3. The Generic Process Framework

A generic software process framework commonly includes five activities:

1. Communication: Work with customers and stakeholders to understand the goals, needs, and constraints of the proposed system.

2. Planning: Estimate effort, cost, time, resources, and risks. Create a plan that guides the project.

3. Modeling: Analyze requirements and create models or designs that describe the software and its structure.

4. Construction: Implement the design by writing code and performing tests to find defects.

5. Deployment: Deliver the software to users, collect feedback, and use that feedback to guide improvements.

4. Umbrella Activities

Umbrella activities are performed throughout the software process rather than in only one stage. They help the team manage and control the project and maintain product quality.

Examples include:
• Software project tracking and control.
• Risk management.
• Software quality assurance.
• Technical reviews.
• Measurement and metrics.
• Software configuration management.
• Reuse management.
• Preparation and production of work products and documentation.

These activities support the main framework activities and help keep development organized.

5. Prescriptive Process Models

Prescriptive process models define an organized structure for software development. They describe a sequence or arrangement of activities and provide guidance about the work products produced during development. They are useful when a project needs clear planning, coordination, and control.

6. The Waterfall Model

The waterfall model organizes development into sequential phases. A typical sequence includes requirements analysis, design, implementation, testing, deployment, and maintenance.

In this model, a phase is generally completed and documented before the next phase begins. This can make progress easier to track and can be suitable when requirements are stable and well understood.

Advantages: Clear stages, defined documentation, and straightforward project tracking.
Limitations: Changes can be expensive later in the project, and users may not see a working product until relatively late.

7. Incremental Process Models

An incremental model develops and delivers the software in a series of smaller releases. Each increment adds useful functionality to the product. The first release provides a basic version, and later releases expand or improve it.

Advantages: Early delivery of useful features, regular feedback, and the ability to prioritize important functions.
Limitations: The system needs a sound overall architecture, and coordinating increments can become difficult if planning is weak.

8. Evolutionary Process Models

Evolutionary models are designed for situations in which requirements are not fully understood at the beginning or are expected to change. The software evolves through repeated versions, using feedback and learning from earlier results.

Prototyping: A prototype is an early, simplified version of a system. It helps users and developers explore requirements, clarify expectations, and evaluate possible designs. A prototype may be discarded or developed further, depending on its purpose.

Spiral Model: The spiral model combines iterative development with explicit risk analysis. Each cycle typically involves setting objectives, identifying and analyzing risks, developing and evaluating the next version, and planning the next cycle. It is especially useful for large or high-risk projects, although it requires expertise in risk assessment.

9. Concurrent Development Model

The concurrent development model represents software activities as states that can occur and progress in parallel. For example, requirements analysis, design, and implementation may be active at the same time for different parts of a system.

This model can reflect the reality of complex projects where teams work on multiple components simultaneously. It requires strong communication and coordination so that changes in one activity do not create unexpected problems in another.

10. Specialized Process Models

Some projects benefit from process models that focus on particular development needs.

Component-Based Development: The system is assembled using reusable software components where possible. The team identifies suitable components, adapts them, and integrates them into the application. Reuse can reduce development effort, but components must be compatible and meet quality requirements.

Formal Methods Model: This approach uses mathematical specification and reasoning to describe and verify system behavior. It can be valuable for systems where correctness and safety are especially important, but it requires specialized skills and can be costly.

Aspect-Oriented Software Development: This approach helps separate concerns that affect multiple parts of a system, such as logging, security, or transaction handling. It aims to make these cross-cutting concerns easier to manage.

11. The Unified Process

The Unified Process is an iterative and incremental framework. Work is organized into repeated iterations, and each iteration produces an improved version of the system. It commonly describes four phases:

1. Inception: Define the project vision, scope, major requirements, and initial feasibility.
2. Elaboration: Refine requirements, address major risks, and establish the system architecture.
3. Construction: Build the product through a series of development iterations and tests.
4. Transition: Prepare the product for users, release it, gather feedback, and resolve remaining issues.

The Unified Process is use-case driven, architecture-centered, and iterative. The amount of work in each phase depends on the needs of the project.

12. Choosing an Appropriate Process Model

A team should select a process model after considering the characteristics of the project. Important questions include:
• Are the requirements clear and stable, or are they likely to change?
• How large and complex is the system?
• How serious are the technical and business risks?
• How often can customers or users provide feedback?
• Does the project require frequent releases?
• Are there regulatory, safety, or documentation requirements?

For example, a project with stable requirements and strict documentation needs may use a more plan-driven approach. A project where users are still discovering what they need may benefit from prototyping or incremental development. High-risk projects may require repeated risk analysis.

13. Practical Project Example: Online Shopping System

Project Description:
A team is developing an online shopping website. Customers must be able to create accounts, browse and search for products, add items to a cart, place orders, pay securely, and track deliveries.

Applying an Incremental Model:
Increment 1 — Basic Store: Build the product catalogue and product details pages.
Increment 2 — Customer Accounts: Add registration, login, and profile management.
Increment 3 — Shopping Cart and Orders: Allow customers to manage a cart and submit orders.
Increment 4 — Payment and Tracking: Integrate a secure payment service and order tracking.
Testing and Feedback: Test each increment, invite users to evaluate the features, and use their feedback to improve later releases.

This approach allows the team to deliver useful functionality in stages instead of waiting until every feature is complete. The team must still plan the overall architecture and pay attention to security, especially when handling customer and payment information.

14. Advantages of Using a Process Model

Using an appropriate process model can provide several benefits:
• It gives the team a shared structure and clear responsibilities.
• It improves planning and progress monitoring.
• It helps identify risks and quality problems earlier.
• It supports communication among developers, customers, and managers.
• It makes project activities and deliverables easier to understand.
• It can help the team respond to change in a controlled way.

15. Conclusion

Process models are important because they provide different ways to organize software development. The waterfall model emphasizes sequential phases, the incremental model delivers functionality in stages, and evolutionary models support learning and change through repeated versions. The concurrent model recognizes that activities can overlap, while specialized models address needs such as component reuse, formal verification, and cross-cutting concerns. The Unified Process combines iterative development with defined project phases.

The main lesson of this chapter is that a process model should be selected according to the needs, risks, and constraints of the project. A thoughtful process helps a team develop software more effectively, but it must be applied with judgment and adapted to the situation.

References

Pressman, R. S., & Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach* (9th Edition). McGraw-Hill Education.

This project is an educational summary of the chapter's process-model concepts, written in original wording; it is not a reproduction of the textbook.